

Strukturalni paterni

Za naš projekat koristit ćemo Facade i Decorator pattern-e.

Facade Pattern

Opis:

Prikazivanje detalja o seriji ili filmu u sistemu zahtijeva podatke iz više povezanih klasa: Sezona, Epizoda, Uloga, Glumac i Recenzija. Bez fasade, klijent bi morao direktno upravljati svim ovim objektima, što stvara preveliku kompleksnost i zavisnost.

Primjena u sistemu:

Uvodi se klasa **KatalogFacade** koja klijentu nudi jednu metodu (npr. **dohvatiSveDetalje(id)**). Ona interno upravlja radom svih navedenih klasa i klijentu vraća spreman set podataka, sakrivajući složenost unutrašnjih veza sistema.

Decorator Pattern

Opis:

Dinamičko dodavanje osobina oglasima (poput vizuelnih stilova ili prioritetnog rangiranja) bez kreiranja velikog broja podklasa za svaku moguću kombinaciju osobina.

Primjena u sistemu:

Uvodi se **OglasDecorator** hijerarhija. Osnovna klasa Oglas ostaje fokusirana na podatke, dok konkretni dekoratori (npr. **PremiumOglasDecorator**) proširuju ponašanje metode **prikaziOglas()** dodajući joj specifične vizuelne ili logičke attribute u zavisnosti od potreba kampanje.

Preostali paterni koje nećemo koristiti:

Adapter Pattern

Opis:

U BMDb sistemu Adapter pattern bi se mogao koristiti za povezivanje sistema notifikacija sa različitim načinima slanja obavijesti korisnicima. Trenutni model koristi klasu Notifikacija, ali različiti servisi za slanje poruka mogu imati različite metode i formate komunikacije.

Primjena u sistemu:

Mogao bi se uvesti interfejs INotifikacijaServis sa metodom poput posaljiNotifikaciju(korisnik, tekst). Zatim bi se napravio EmailNotifikacijaAdapter, koji bi prilagodio eksterni email servis načinu rada BMDb sistema. Na primjer, kada administrator objavi novu seriju ili film, sistem bi pozvao jedinstvenu metodu za slanje notifikacije, dok bi adapter interno komunicirao sa email servisom i pripremao odgovarajući format poruke.

Na isti način mogli bi se dodati i:

SMSNotifikacijaAdapter

PushNotifikacijaAdapter

bez promjene ostatka aplikacije.

Zašto se ne koristi:

Ovaj patern nećemo koristiti jer BMDb trenutno koristi jednostavan sistem notifikacija unutar same aplikacije i nema integraciju sa vanjskim servisima za email, SMS ili push obavijesti.

Bridge Pattern

Opis:

Bridge pattern bi se u BMDb sistemu mogao koristiti kada bi se odvojio sadržaj koji se prikazuje od načina njegovog prikaza. Na primjer, klase Film i Serija predstavljaju apstrakciju sadržaja, dok bi različite implementacije prikaza mogle biti web prikaz, mobilni prikaz ili API prikaz.

Primjena u sistemu:

Mogla bi se uvesti apstrakcija EntertainmentPrikaz, koja koristi interfejs IPrikazImplementacija. Konkretno implementacije mogle bi biti WebPrikaz, MobilniPrikaz i ApiPrikaz. Tako bi se Film i Serija mogli prikazivati kroz različite kanale bez mijenjanja osnovnih modela.

Zašto se ne koristi:

Ovaj patern nećemo koristiti jer BMDb trenutno koristi klasični MVC web prikaz i nema potrebu za više nezavisnih implementacija prikaza.

Composite pattern

Opis:

Composite pattern bi se mogao koristiti za modelovanje hijerarhije sadržaja u BMDb sistemu. Najbolji primjer je Serija, koja se sastoji od više sezona, a svaka sezona može imati više epizoda. Tu postoji prirodna struktura tipa cjelina–dio.

Primjena u sistemu:

Mogao bi se uvesti zajednički interfejs ISadrzajKomponenta sa metodama poput prikaziDetalje() ili izracunajTrajanje(). Klase Serija, Sezona i Epizoda mogle bi implementirati isti interfejs, pa bi sistem mogao jednako tretirati cijelu seriju, jednu sezonu ili pojedinačnu epizodu.

Zašto se ne koristi:

Ovaj patern nećemo koristiti jer trenutni model nema posebno implementiranu klasu Epizoda, a postojeća klasa Sezona je dovoljno jednostavna i ne zahtijeva potpunu hijerarhijsku strukturu.

Proxy pattern

Opis:

U BMDb sistemu Proxy pattern bi se mogao koristiti za kontrolu pristupa funkcionalnostima koje nisu dostupne svim korisnicima. Obični korisnik može pregledati filmove, serije, trailere i ostavljati recenzije, ali ne smije upravljati opcijama koje pripadaju moderatorima i administratorima.

Primjena u sistemu:

Mogao bi se uvesti UpravljanjeSistemomProxy, koji bi stajao između korisnika i osjetljivih funkcionalnosti sistema. Prije izvršavanja akcija kao što su dodavanje filma, brisanje recenzije, upravljanje korisnicima ili upravljanje oglasima, proxy bi provjerio status korisnika.

Na primjer:

ako je korisnik **Moderator**, dozvoljava mu se upravljanje sadržajem i recenzijama;
ako je korisnik **Administrator**, dozvoljava mu se upravljanje korisnicima, moderatorima i oglasima;
ako je korisnik obični **Korisnik**, pristup tim opcijama se odbija.

Zašto se ne koristi:

Ovaj patern nećemo koristiti kao poseban sloj jer se u **ASP.NET MVC** projektu kontrola pristupa već može riješiti kroz role, autorizaciju i provjere u kontrolerima. Ipak, Proxy pattern bi bio koristan ako bismo željeli dodatno centralizovati provjeru prava pristupa u posebnu klasu.